**Abdul Munam Aziz**
Roll No: 23L-3019

# Architecture Overview

Data source: Frankfurter API (https://api.frankfurter.dev/v1/latest), base currency EUR, targets USD/GBP/JPY.

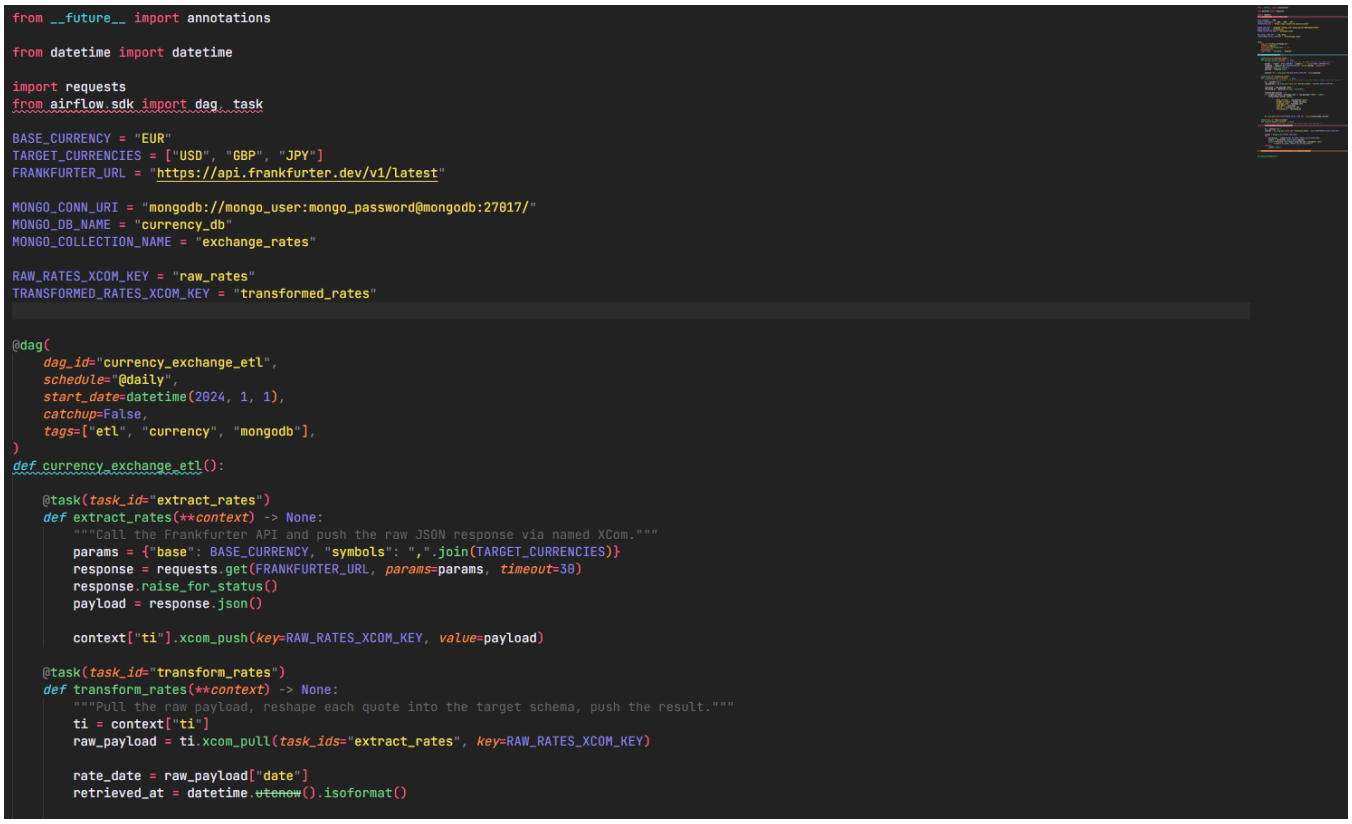
Orchestration: Apache Airflow, run locally via the Astro CLI (astro dev start).

Storage: MongoDB 7.0, with Mongo Express as a web-based admin UI.

Both MongoDB and Mongo Express run as additional Docker services alongside Airflow's own containers, sharing the same Docker network so the DAG can reach MongoDB by its service hostname (mongodb). MongoDB data persists in a named Docker volume (mongodb_data), surviving container restarts.

# Code

## DAG: extract_rates and transform_rates



```python
from __future__ import annotations

from datetime import datetime

import requests
from airflow.sdk import dag, task

BASE_CURRENCY = "EUR"
TARGET_CURRENCIES = ["USD", "GBP", "JPY"]
FRANKFURTER_URL = "https://api.frankfurter.dev/v1/latest"

MONGO_CONN_URI = "mongodb://mongo_user:mongo_password@mongodb:27017/"
MONGO_DB_NAME = "currency_db"
MONGO_COLLECTION_NAME = "exchange_rates"

RAW_RATES_XCOM_KEY = "raw_rates"
TRANSFORMED_RATES_XCOM_KEY = "transformed_rates"


@dag(
    dag_id="currency_exchange_etl",
    schedule="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["etl", "currency", "mongodb"],
)
def currency_exchange_etl():

    @task(task_id="extract_rates")
    def extract_rates(**context) -> None:
        """Call the Frankfurter API and push the raw JSON response via named XCom."""
        params = {"base": BASE_CURRENCY, "symbols": ",".join(TARGET_CURRENCIES)}
        response = requests.get(FRANKFURTER_URL, params=params, timeout=30)
        response.raise_for_status()
        payload = response.json()

        context["ti"].xcom_push(key=RAW_RATES_XCOM_KEY, value=payload)

    @task(task_id="transform_rates")
    def transform_rates(**context) -> None:
        """Pull the raw payload, reshape each quote into the target schema, push the result."""
        ti = context["ti"]
        raw_payload = ti.xcom_pull(task_ids="extract_rates", key=RAW_RATES_XCOM_KEY)

        rate_date = raw_payload["date"]
        retrieved_at = datetime.utcnow().isoformat()
```

*currency_etl_dag.py — imports, configuration constants, extract_rates and transform_rates tasks.*

## DAG: transform_rates (continued) and load_to_mongo

```
        transformed_records = []
        for target_currency, exchange_rate in raw_payload["rates"].items():
            transformed_records.append(
                {
                    "base_currency": raw_payload["base"],
                    "target_currency": target_currency,
                    "exchange_rate": exchange_rate,
                    "rate_date": rate_date,
                    "source": "Frankfurter API",
                    "retrieved_at": retrieved_at,
                }
            )

        ti.xcom_push(key=TRANSFORMED_RATES_XCOM_KEY, value=transformed_records)

    @task(task_id="load_to_mongo")
    def load_to_mongo(**context) -> None:
        """Pull the transformed records and insert them into MongoDB."""
        from pymongo import MongoClient

        ti = context["ti"]
        records = ti.xcom_pull(task_ids="transform_rates", key=TRANSFORMED_RATES_XCOM_KEY)

        client = MongoClient(MONGO_CONN_URI)
        try:
            collection = client[MONGO_DB_NAME][MONGO_COLLECTION_NAME]
            result = collection.insert_many(records)
            print(f"Inserted {len(result.inserted_ids)} documents into "
                  f"{MONGO_DB_NAME}.{MONGO_COLLECTION_NAME}")
        finally:
            client.close()

    extract_rates() >> transform_rates() >> load_to_mongo()
      You, 19 minutes ago • Add extract_rates task: fetch exchange rates fr…

currency_exchange_etl()
```

*currency_etl_dag.py — transform_rates XCom push, load_to_mongo task, and task dependency chain (extract_rates() >> transform_rates() >> load_to_mongo()).*

## docker-compose.override.yml



```yaml
# docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic applications (compose-spec.json) | ▷ Run All Services | You, 19 minutes ago | author (You)
services:
  # ▷ Run Service
  mongodb:
    image: mongo:7.0
    container_name: currency-etl-mongodb
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: mongo_user
      MONGO_INITDB_ROOT_PASSWORD: mongo_password
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db
    networks:
      - airflow

  # ▷ Run Service
  mongo-express:
    image: mongo-express:1.0.2
    container_name: currency-etl-mongo-express
    restart: unless-stopped
    depends_on:
      - mongodb
    environment:
      ME_CONFIG_MONGODB_SERVER: mongodb
      ME_CONFIG_MONGODB_PORT: 27017
      ME_CONFIG_MONGODB_ADMINUSERNAME: mongo_user
      ME_CONFIG_MONGODB_ADMINPASSWORD: mongo_password
      ME_CONFIG_BASICAUTH_USERNAME: admin
      ME_CONFIG_BASICAUTH_PASSWORD: admin_password
    ports:
      - "8081:8081"
    networks:
      - airflow

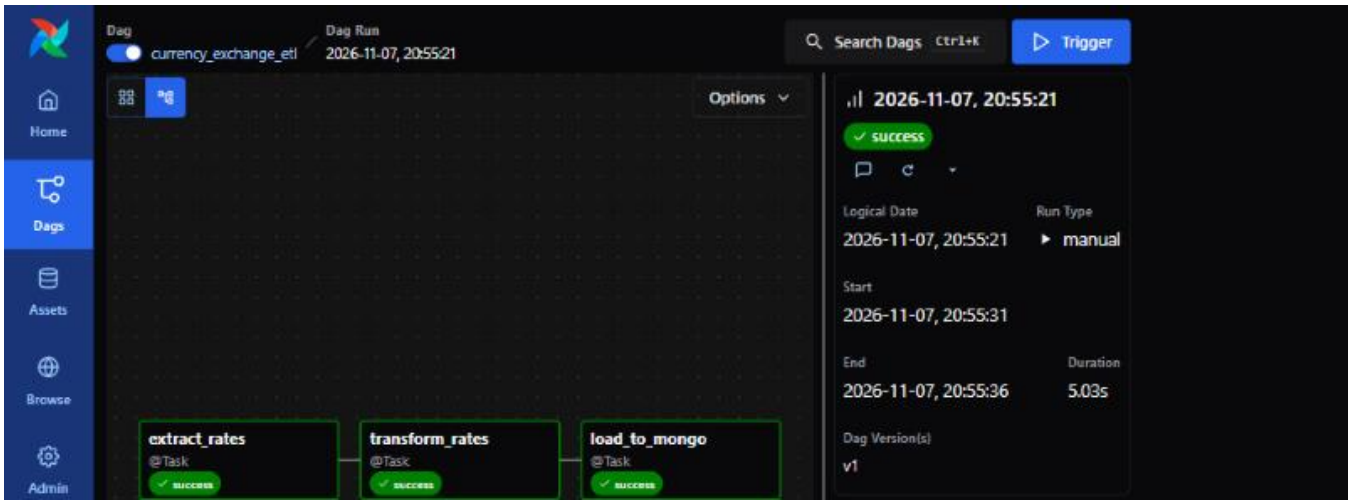
volumes:
  mongodb_data:

networks:
  airflow:
    name: airflow-currency-etl_e837e7_airflow
    external: true      # You, 19 minutes ago • Add MongoDB and Mongo Express Docker services w…
```

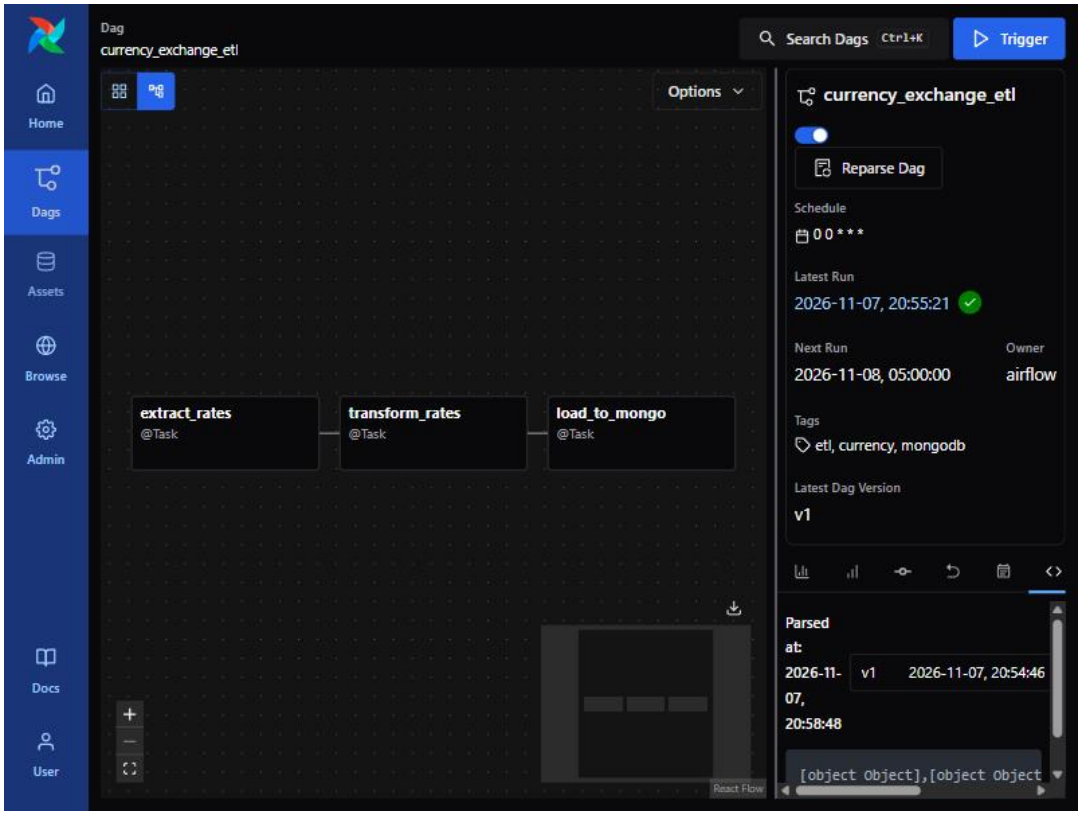
*docker-compose.override.yml — MongoDB and Mongo Express service definitions, persistent volume, and external Airflow network attachment.*

# Execution Results

## DAG run — all tasks succeeded




## DAG graph structure

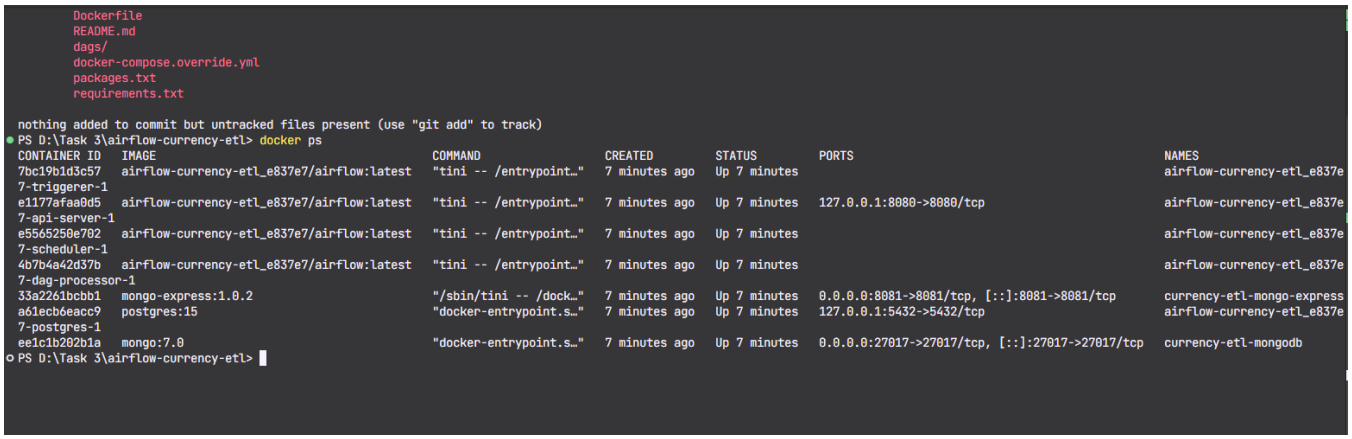



*Figures above: Airflow UI graph views of currency_exchange_etl, confirming all three tasks (extract_rates, transform_rates, load_to_mongo) succeeded in the linear dependency chain.*

## load_to_mongo task log

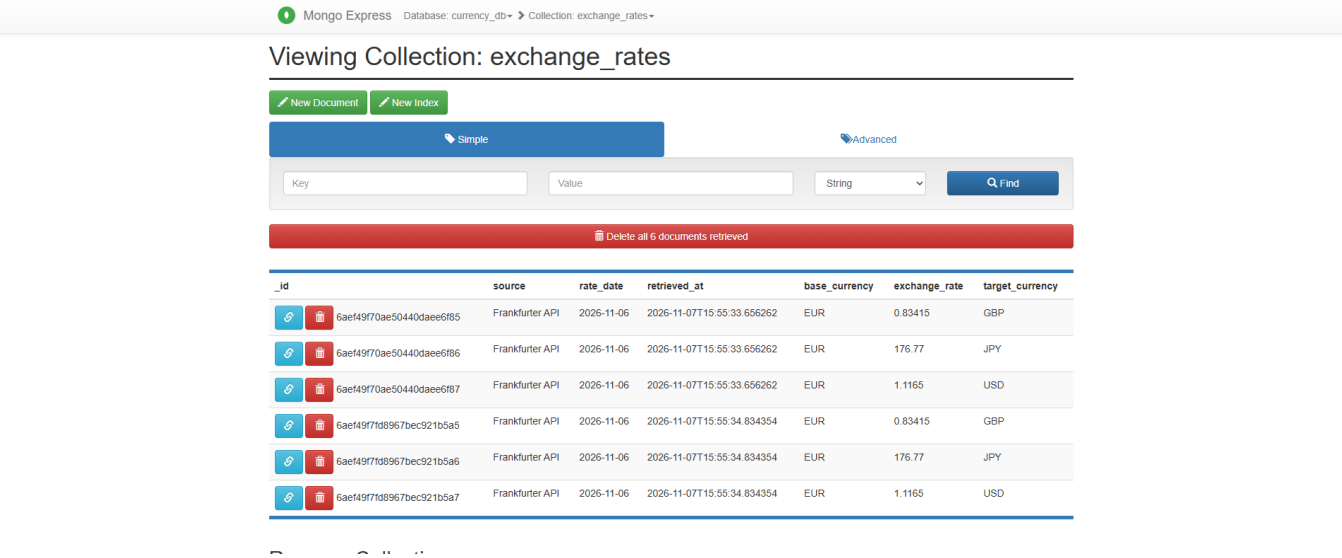



## Running containers




*Figures above: the load_to_mongo log confirming "Inserted 3 documents into currency_db.exchange_rates", and docker ps showing all 7 containers (Airflow, Postgres, MongoDB, Mongo Express) running together.*

## Data verified in Mongo Express




*Mongo Express UI showing the exchange_rates collection inside currency_db, with documents matching the required schema (base_currency, target_currency, exchange_rate, rate_date, source, retrieved_at).*

# Source Code Repository

The complete project (DAG, Dockerfile, Docker Compose service definitions, and this report) has been pushed to GitHub as a sequence of incremental commits, one per subtask:

https://github.com/Munam32/airflow-currency-etl